Lista de exercícios – Dicionários

Orientações para resolução da lista

- Esta lista trabalha exclusivamente o conteúdo de **dicionários**: criação, acesso pela chave, alteração de valores, inclusão de novas associações, verificação de existência com `in`, remoção com `del`, uso de `keys()`, `values()` e `items()` e percurso com `for`.
- Todas as questões são de **construção de código**. Antes de implementar qualquer solução, procure identificar claramente as entradas, o processamento necessário e as saídas esperadas.
- Lembre-se de que a mesma sintaxe pode representar operações diferentes: `d[chave] = valor` **altera** o valor quando a chave já existe e **adiciona** uma nova associação quando ela ainda não existe.
- Acessar uma chave inexistente provoca erro. Sempre que houver dúvida, verifique a existência da chave antes de utilizá-la.
- Em cada problema, decida primeiro o que será a chave e o que será o valor. Essa escolha define como os dados serão consultados e atualizados.
- Teste cada solução com os exemplos do enunciado e elabore também os seus próprios casos de teste.
- Desenvolva as soluções individualmente. O objetivo da lista é consolidar seu aprendizado e desenvolver autonomia na construção de programas.

Exercícios

1. Imagine um sistema simples de personagens de um jogo. Faça um programa que leia o nome de um personagem e crie um dicionário com as chaves `"nome"`, `"vida"` e `"nivel"`, iniciando a vida com 100 e o nível com 1. Em seguida, o programa deve:

a) exibir os dados iniciais do personagem;

b) ler um valor de dano e subtraí-lo da vida;

c) aumentar o nível em uma unidade, caso a vida continue maior que zero;

d) adicionar a chave `"status"` com o valor `"ativo"` se a vida for maior que zero, ou `"derrotado"` caso contrário;

e) ler o nome de um equipamento e adicioná-lo ao dicionário;

f) exibir todos os dados atualizados, um por linha, no formato `chave: valor`.

Por exemplo, para o personagem `Arqueira`, dano `30` e equipamento `Arco`, a exibição final deve ser:

```
nome: Arqueira
vida: 70
nivel: 2
status: ativo
equipamento: Arco
```

2. Uma loja controla seu estoque por meio do seguinte dicionário:

```
estoque = {"mouse": 10, "teclado": 5, "monitor": 2}
```

Faça um programa que registre uma venda. O programa deve ler o nome de um produto e a quantidade vendida e, em seguida:

- exibir `Produto inexistente` se o produto não fizer parte do estoque;
- exibir `Estoque insuficiente` se a quantidade pedida for maior que a disponível;
- atualizar a quantidade do produto nos demais casos. Se a quantidade chegar a zero, o produto deve ser removido do dicionário.

Ao final, exiba todos os produtos restantes no formato `produto -> quantidade unidades` e o total de itens armazenados.

Por exemplo, para a venda de 2 unidades de `monitor`, a saída deve ser:

```
mouse -> 10 unidades
teclado -> 5 unidades
Total: 15
```

3. Faça um programa que realize a apuração de uma votação. O programa deve ler nomes de candidatos, um por linha, até que seja digitado `fim`, utilizando um dicionário para contabilizar quantos votos cada candidato recebeu.

Ao final, exiba cada candidato com sua quantidade de votos e seu percentual em relação ao total, e informe o vencedor da votação. Em caso de empate, considere o candidato que recebeu seu primeiro voto antes.

Por exemplo, para os votos:

```
Ana
Carlos
Ana
Julia
Ana
fim
```

a saída deve indicar:

```
Ana: 3 votos (60.0%)
Carlos: 1 votos (20.0%)
Julia: 1 votos (20.0%)
Vencedor: Ana
```

Caso nenhum voto seja informado, exiba apenas a mensagem `Nenhum voto registrado`.

4. Uma plataforma armazena seu catálogo de filmes em uma lista de dicionários. Cada filme possui um título e uma nota:

```
filmes = [
    {"titulo": "Horizonte", "nota": 8.5},
    {"titulo": "Orbita",    "nota": 6.0},
    {"titulo": "Labirinto", "nota": 9.0},
    {"titulo": "Mare Alta", "nota": 7.5}
]
```

Crie uma função chamada `filmes_recomendados(filmes, nota_minima)` que receba a lista de filmes e uma nota e retorne uma lista com os títulos dos filmes cuja nota é maior ou igual à nota informada.

Para o exemplo acima, `filmes_recomendados(filmes, 8.0)` deve retornar:

```
['Horizonte', 'Labirinto']
```

No programa principal, leia a quantidade de filmes e os dados de cada um, montando a lista de dicionários. Em seguida, exiba o filme com a maior nota, o filme com a menor nota, a média das avaliações e os títulos recomendados para uma nota mínima informada pelo usuário.

5. Uma loja mantém um dicionário no qual cada chave é o nome de um produto e cada valor é a categoria à qual ele pertence:

```
produtos = {
    "mouse": "perifericos",
    "monitor": "video",
    "teclado": "perifericos",
    "webcam": "video",
    "cadeira": "moveis"
}
```

Crie uma função chamada `organizar_por_categoria(produtos)` que retorne um novo dicionário organizado por categoria. Cada chave deve representar uma categoria, e o valor correspondente deve ser a lista com os nomes dos produtos daquela categoria.

Para o exemplo acima, a função deve retornar:

```
{'perifericos': ['mouse', 'teclado'], 'video': ['monitor', 'webcam'],
 'moveis': ['cadeira']}
```

No programa principal, utilize o dicionário retornado para exibir todas as categorias cadastradas, os produtos de cada categoria e a quantidade de produtos existentes em cada grupo.

6. Uma loja deseja analisar os preços de seus produtos. Faça um programa que leia um número inteiro N e, em seguida, o nome e o preço de N produtos, armazenando-os em um dicionário no qual a chave é o nome do produto e o valor é o seu preço.

Ao final, o programa deve exibir a média dos preços, o produto mais caro, o produto mais barato e todos os produtos com preço acima da média, no formato `produto -> preco`.

Por exemplo, para o dicionário:

```
{"mouse": 120.0, "teclado": 200.0, "monitor": 900.0, "cabo": 20.0}
```

a saída deve indicar:

```
Media: 310.0
Mais caro: monitor
Mais barato: cabo
monitor -> 900.0
```

Em caso de empate, considere o primeiro produto cadastrado. Não utilize as funções `max()`, `min()` e `sum()`.

7. Crie uma função chamada `contar_letras(palavra)` que receba uma string e retorne um dicionário no qual cada chave é uma letra da palavra e cada valor é a quantidade de vezes que essa letra apareceu.

Por exemplo:

```
contar_letras("banana") -> {'b': 1, 'a': 3, 'n': 2}
contar_letras("ovo")    -> {'o': 2, 'v': 1}
contar_letras("")       -> {}
```

No programa principal, leia uma palavra, chame a função e exiba a letra mais frequente, a quantidade de ocorrências dessa letra e a quantidade de letras diferentes presentes na palavra.

Caso duas ou mais letras possuam a mesma frequência máxima, deve ser escolhida aquela que apareceu primeiro na palavra. Não utilize o método `count()`.

8. Uma rede possui duas lojas, e cada uma mantém seu estoque em um dicionário no qual a chave é o nome do produto e o valor é a quantidade disponível.

Crie uma função chamada `unir_estoques(loja1, loja2)` que receba os dois dicionários e retorne um **novo** dicionário representando o estoque total da rede. Produtos presentes nas duas lojas devem ter suas quantidades somadas, e produtos presentes em apenas uma delas devem aparecer com a quantidade original.

Por exemplo:

```
loja1 = {"mouse": 10, "teclado": 5, "monitor": 2}
loja2 = {"teclado": 3, "cadeira": 15, "mouse": 1}
```

deve produzir:

```
{'mouse': 11, 'teclado': 8, 'monitor': 2, 'cadeira': 15}
```

Os dois dicionários recebidos como parâmetro não devem ser modificados.

No programa principal, utilize o dicionário retornado para remover da rede todos os produtos cuja quantidade total seja menor que 5 e exiba o estoque resultante. Atenção: não remova itens de um dicionário enquanto ele estiver sendo percorrido. Guarde primeiro, em uma lista, as chaves que deverão ser removidas.

9. Uma empresa possui funcionários distribuídos em diferentes setores. Os dados são armazenados em uma lista de dicionários, em que cada funcionário possui nome, setor e salário.

Por exemplo:

```
funcionarios = [
    {"nome": "Ana",   "setor": "TI",     "salario": 5200.0},
    {"nome": "Bruno", "setor": "Vendas", "salario": 3100.0},
```

```
    {"nome": "Carla", "setor": "TI",     "salario": 6800.0},
    {"nome": "Davi",  "setor": "Vendas", "salario": 2900.0},
    {"nome": "Eva",   "setor": "RH",     "salario": 4000.0}
]
```

Crie uma função chamada `agrupar_por_setor(funcionarios)` que percorra essa estrutura e retorne um novo dicionário organizado por setor. Cada chave deve representar um setor, e o valor correspondente deve ser a lista com os salários dos funcionários daquele setor, na ordem em que aparecem na lista original.

Para o exemplo acima, a função deve retornar:

```
{'TI': [5200.0, 6800.0], 'Vendas': [3100.0, 2900.0], 'RH': [4000.0]}
```

No programa principal, utilize o dicionário retornado para exibir, para cada setor, a quantidade de funcionários e a média salarial. Informe também o setor com a maior média salarial e o nome do funcionário com o maior salário da empresa. Em caso de empate, considere o primeiro encontrado.

10. Uma biblioteca controla seu acervo e seus empréstimos utilizando dois dicionários. O primeiro associa o título de cada livro à quantidade de exemplares disponíveis. O segundo associa o nome de cada leitor à lista de títulos que estão atualmente com ele.

Considere que o acervo inicial é:

```
acervo = {
    "Algoritmos": 2,
    "Python": 1,
    "Redes": 3
}
```

e que o dicionário de empréstimos começa vazio.

Faça um programa que leia uma quantidade indeterminada de operações. Cada operação é composta por três informações, lidas uma por linha: o tipo da operação (`emprestar` ou `devolver`), o nome do leitor e o título do livro. A leitura termina quando o tipo informado for `fim`.

Em um **empréstimo**, o programa deve recusar a operação, exibindo uma mensagem adequada, quando:

- o título não existir no acervo;
- não houver exemplares disponíveis;
- o leitor já estiver com 2 livros;
- o leitor já estiver com um exemplar do mesmo título.

Caso contrário, a quantidade disponível deve ser reduzida e o título deve ser acrescentado à lista do leitor. Um leitor que ainda não possui registro deve ser incluído no dicionário de empréstimos no momento de seu primeiro empréstimo.

Em uma **devolução**, o programa deve recusar a operação quando o leitor não possuir registro ou não estiver com o título informado. Caso contrário, a quantidade disponível deve ser aumentada e o título deve ser retirado da lista do leitor. Quando a lista de um leitor ficar vazia, ele deve ser removido do dicionário de empréstimos.

Ao encerrar a leitura, o programa deve exibir:

a) o acervo atualizado, no formato `titulo -> quantidade`;

b) cada leitor com empréstimos em aberto e os títulos que estão com ele;

c) a quantidade total de exemplares emprestados;

d) os títulos que estão com todos os exemplares emprestados.

Para retirar um título da lista de um leitor, construa uma nova lista contendo os demais títulos.